Inteligência Artificial

Trabalho T1 — Busca Competitiva

Connect 4 (Lig 4) com Minimax e Poda Alfa-Beta

Integrantes do grupo:

[NOME DO ALUNO 1]

[NOME DO ALUNO 2]

[NOME DO ALUNO 3]

Professora: [NOME DA PROFESSORA]

[CIDADE], outubro de 2026

1. O jogo escolhido: Connect 4

O Connect 4 (no Brasil, também chamado de Lig 4) é um jogo de tabuleiro para dois jogadores. O tabuleiro fica em pé, tem 6 linhas e 7 colunas, e as peças são soltas por cima de uma coluna e caem até a posição livre mais baixa, como se houvesse gravidade.

1.1 Regras

1. Cada jogador tem peças de uma cor. Na nossa implementação, o humano joga com **X** e a IA com **O**.
2. Os jogadores se alternam. Em cada vez, o jogador escolhe uma coluna que ainda tenha espaço e solta uma peça nela.
3. A peça ocupa a casa vazia mais baixa da coluna escolhida. Não é possível jogar em uma coluna cheia.
4. Vence quem primeiro alinhar **4 peças seguidas** da mesma cor na horizontal, na vertical ou em uma das diagonais.
5. Se as 42 casas forem preenchidas sem que ninguém alinhe 4 peças, a partida termina **empatada**.

2. Formulação do problema de busca

Em busca competitiva, o problema é formulado como um jogo com dois agentes adversários: **MAX** (a IA, que quer maximizar o resultado) e **MIN** (o humano, que quer minimizá-lo). A formulação fica assim:

Elemento	Definição no Connect 4
Estado	Matriz 6×7 em que cada casa vale 0 (vazia), 1 (peça do humano) ou 2 (peça da IA). De quem é a vez não fica guardado na matriz: durante a busca, isso é indicado pelo nível da árvore (MAX ou MIN) e, na partida, por uma variável do laço principal.
Estado inicial	Tabuleiro com as 42 casas vazias. O primeiro a jogar é escolhido no início da partida (humano ou IA).
Jogador(s)	Define de quem é a vez. Os jogadores se alternam: MAX (IA) e MIN (humano).
Ações / função sucessor	Ações(s) = colunas c ∈ {1..7} cuja casa do topo está vazia. Resultado(s, c) = novo tabuleiro com a peça do jogador da vez na linha livre mais baixa da coluna c. São no máximo 7 sucessores por estado.
Estado objetivo (teste terminal)	O estado é terminal quando um jogador tem 4 peças alinhadas (horizontal, vertical ou diagonal) ou quando o tabuleiro está cheio. O objetivo da IA é chegar a um estado terminal em que ela venceu.
Função utilidade	+1.000.000 se a IA venceu, −1.000.000 se o humano venceu e 0 em caso de empate. Somamos a profundidade restante para que a IA prefira vencer mais rápido e, quando perder, perder o mais tarde possível.
Custo de caminho	Cada jogada custa 1 (custo uniforme). Em jogos, porém, o custo do caminho não é o critério de decisão; o que importa é o valor de utilidade do estado terminal. Por isso o custo só aparece indiretamente, na preferência por vitórias mais curtas.

O espaço de estados é grande: o Connect 4 tem cerca de $4{,}5 \times 10^{12}$ posições válidas. Com fator de ramificação máximo 7 e partidas de até 42 jogadas, não é viável explorar a árvore inteira em cada jogada. Por isso a busca é limitada a uma profundidade e os estados não terminais nesse limite são avaliados por uma **função heurística**.

3. Tipo de jogo: determinístico

O Connect 4 é um jogo **determinístico**: não existe nenhum elemento de sorte, como dados, cartas sorteadas ou eventos aleatórios. Dado um estado e uma ação, o próximo estado é sempre o mesmo, porque a peça sempre cai na mesma casa.

Além disso, o jogo é:

- **de informação perfeita**: os dois jogadores veem o tabuleiro inteiro o tempo todo, e nada fica escondido;
- **de soma zero**: o que é bom para um jogador é ruim, na mesma medida, para o outro (vitória de um é derrota do outro);
- **de dois jogadores, alternado**: as jogadas são feitas uma de cada vez.

Essas características são exatamente as que o algoritmo Minimax supõe. Se o jogo fosse estocástico (por exemplo, gamão, que usa dados), seria necessário incluir nós de acaso na árvore e usar o **Expectiminimax**.

4. Algoritmo escolhido: Minimax com poda alfa-beta

4.1 Por que este algoritmo

O **Minimax** é o algoritmo clássico para jogos determinísticos, de soma zero e com informação perfeita. Ele monta a árvore de jogadas possíveis e supõe que o adversário sempre faz a melhor jogada para ele. Nos níveis da IA (MAX), escolhe o maior valor entre os filhos; nos níveis do humano (MIN), escolhe o menor. O valor que sobe até a raiz indica a melhor jogada considerando que o adversário joga bem.

A **poda alfa-beta** é uma melhoria do Minimax que chega exatamente à mesma decisão, mas sem visitar galhos que não podem mudar o resultado:

- **alfa** guarda o melhor valor que MAX já tem garantido no caminho atual;
- **beta** guarda o melhor valor que MIN já tem garantido no caminho atual;
- quando **alfa ≥ beta**, um dos jogadores já tem uma alternativa melhor em outro galho, então os filhos restantes são descartados (podados).

Para aumentar a poda, as colunas são avaliadas **do centro para as bordas** (4, 3, 5, 2, 6, 1, 7), porque jogadas centrais costumam ser as melhores. Quanto antes a melhor jogada é avaliada, mais cedo aparece o corte.

4.2 Função heurística (avaliação de estados não terminais)

Quando a busca chega à profundidade limite sem fim de jogo, o tabuleiro é avaliado por uma heurística. Ela percorre todas as **janelas** de 4 casas consecutivas (horizontais, verticais e diagonais) e soma pontos do ponto de vista da IA:

Situação na janela de 4 casas	Pontos
3 peças da IA + 1 casa vazia (ameaça de vitória)	+5
2 peças da IA + 2 casas vazias (sequência em construção)	+2
3 peças do humano + 1 casa vazia (ameaça do adversário)	−4
Cada peça da IA na coluna central	+3

A coluna central recebe bônus porque participa do maior número de sequências possíveis de 4.

4.3 Pseudocódigo

```
função minimax(estado, prof, alfa, beta, vez_da_IA):
    se IA venceu:      retorna +1.000.000 + prof
    se humano venceu:  retorna -1.000.000 - prof
    se tabuleiro cheio: retorna 0
    se prof = 0:       retorna heurística(estado)
    se vez_da_IA:                                  # nó MAX
        valor = -infinito
        para cada coluna válida (do centro para as bordas):
            valor = max(valor, minimax(sucessor, prof-1, alfa, beta, falso))
            alfa = max(alfa, valor)
            se alfa >= beta: interrompe (poda)
    senão:                                          # nó MIN
        valor = +infinito
        para cada coluna válida:
            valor = min(valor, minimax(sucessor, prof-1, alfa, beta, verdadeiro))
            beta = min(beta, valor)
            se alfa >= beta: interrompe (poda)
    retorna valor
# prof = profundidade restante da busca
```

4.4 Resultado medido: efeito da poda

Medimos com o próprio código, em um notebook comum, (função escolher_jogada, com e sem poda) quantos estados a IA visita para decidir uma jogada. As duas versões sempre escolheram a mesma coluna, o que confirma que a poda não altera a decisão:

Posição	Profundidade	Sem poda (nós / tempo)	Com poda (nós / tempo)	Redução
Tabuleiro inicial	4	2.801 / 0,13 s	178 / 0,01 s	93,6%
Tabuleiro inicial	6	137.257 / 6,18 s	1.370 / 0,05 s	99,0%
Meio de jogo (6 peças)	4	2.800 / 0,13 s	250 / 0,01 s	91,1%
Meio de jogo (6 peças)	6	132.375 / 6,83 s	3.876 / 0,17 s	97,1%

Sem poda, cada nível a mais multiplica o trabalho por quase 7. Com a poda e a ordenação pelo centro, o nível difícil (profundidade 6) responde, em geral, em menos de 1 segundo. Os tempos variam conforme o computador; as contagens de nós não variam. Nos testes automáticos, a IA também venceu ou empatou todas as 10 partidas contra um jogador que joga ao acaso.

5. Implementação em Python

A solução usa apenas a biblioteca padrão do Python (3.8 ou mais novo). O código está todo comentado e organizado assim:

Arquivo	Responsabilidade
connect4/tabuleiro.py	Estado inicial, função sucessor (jogadas válidas e aplicação de jogada), teste de vitória e de empate.
connect4/ia.py	Minimax com poda alfa-beta, função heurística e níveis de dificuldade (profundidade 2, 4 ou 6).
connect4/main.py	Partida humano × IA no terminal, com validação das jogadas digitadas.
jogar.py	Atalho para iniciar o jogo.
tests/test_connect4.py	16 testes automáticos: regras, vitórias nas 4 direções, empate, entrada inválida e decisões da IA.

Como executar: na pasta do projeto, rode o primeiro comando abaixo, escolha a dificuldade (facil, medio ou dificil) e quem começa. Em cada vez, digite o número da coluna (1 a 7). O segundo comando roda os testes automáticos.

```
python jogar.py
python -m unittest -v
```

6. Reflexão: foi o algoritmo mais indicado?

Para o Connect 4, consideramos que o **Minimax com poda alfa-beta foi uma escolha adequada**. O jogo tem exatamente as características que o algoritmo supõe: é determinístico, tem informação perfeita, é de soma zero e tem dois jogadores. O fator de ramificação é pequeno (no máximo 7), o que permite olhar 6 jogadas à frente em fração de segundo. A poda reduziu os nós visitados em mais de 90% sem mudar a decisão.

A limitação está na **heurística**: com profundidade limitada, a qualidade da IA depende de como avaliamos os tabuleiros que não terminaram. Uma heurística simples pode deixar passar ameaças que só aparecem além do horizonte de busca.

Outros algoritmos que poderiam ser indicados:

- **Negamax com aprofundamento iterativo e tabela de transposição**: é a mesma ideia do Minimax, escrita de forma mais compacta. Guarda posições já avaliadas, que se repetem muito no Connect 4 porque jogadas em ordens diferentes levam ao mesmo tabuleiro, e aprofunda a busca enquanto houver tempo. É a base dos programas que resolvem o

Connect 4 por completo (sabe-se que o primeiro jogador vence se começar pela coluna central).

- **Monte Carlo Tree Search (MCTS)**: em vez de uma heurística escrita à mão, avalia as jogadas simulando muitas partidas aleatórias até o fim. É útil quando é difícil criar uma boa função de avaliação, como no Go, mas para o Connect 4 costuma precisar de mais tempo que o alfa-beta para jogar no mesmo nível.
- **Expectiminimax**: seria o indicado se o jogo tivesse sorte (dados, sorteios). Como o Connect 4 é determinístico, ele não se aplica aqui.

Concluímos que, entre os algoritmos estudados, o Minimax com poda alfa-beta é o mais indicado para o Connect 4. O próximo passo natural para melhorar a IA seria acrescentar uma tabela de transposição e o aprofundamento iterativo.

7. Organização do grupo

Divisão de tarefas e cronograma combinados pelo grupo (preencher):

Integrante	Responsabilidade	Prazo	Situação
[NOME DO ALUNO 1]			
[NOME DO ALUNO 2]			
[NOME DO ALUNO 3]			

Referências

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: uma abordagem moderna**. 4. ed. Rio de Janeiro: GEN LTC, 2022. Cap. 5 — Busca competitiva e jogos.

ALLIS, Louis Victor. **A knowledge-based approach of Connect-Four**. Amsterdam: Vrije Universiteit, 1988. Dissertação (Mestrado).